

CHAP 9: (2) 반복문 While

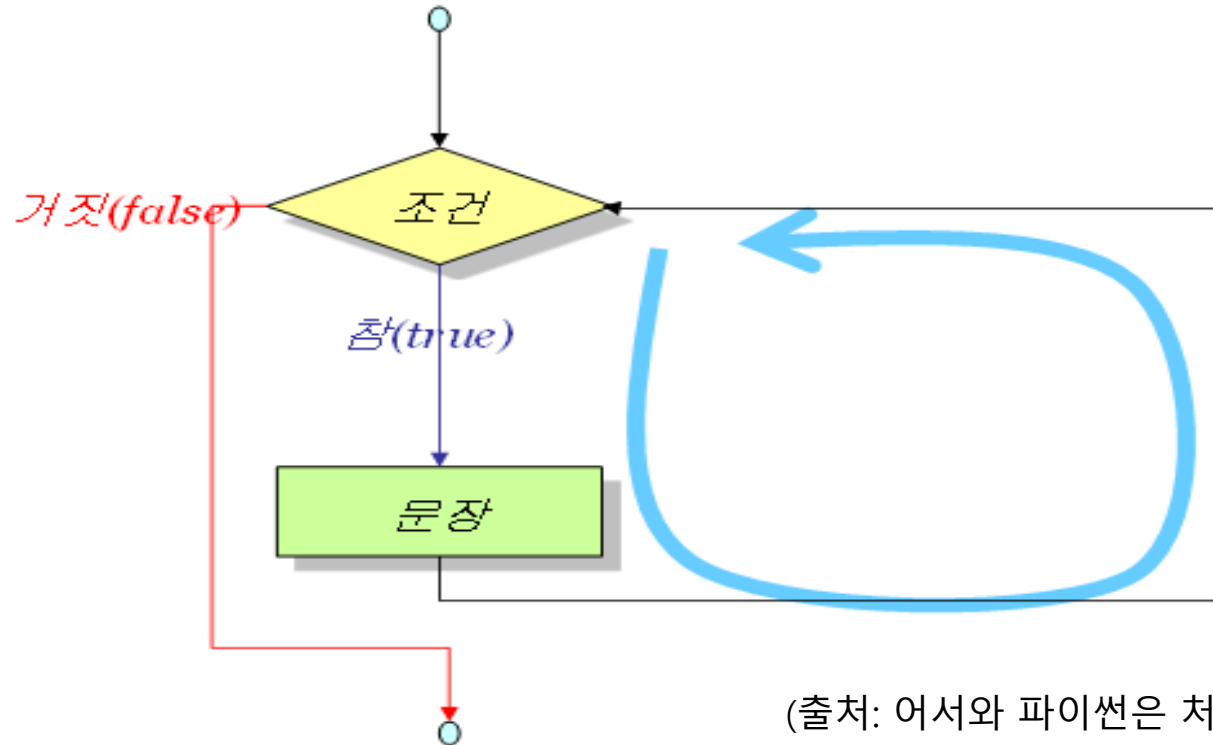
while문

break Statement

continue Statement

while 문

- while 문은 **조건**을 정해놓고 **참(True)**이라면 반복하는 구조이다.

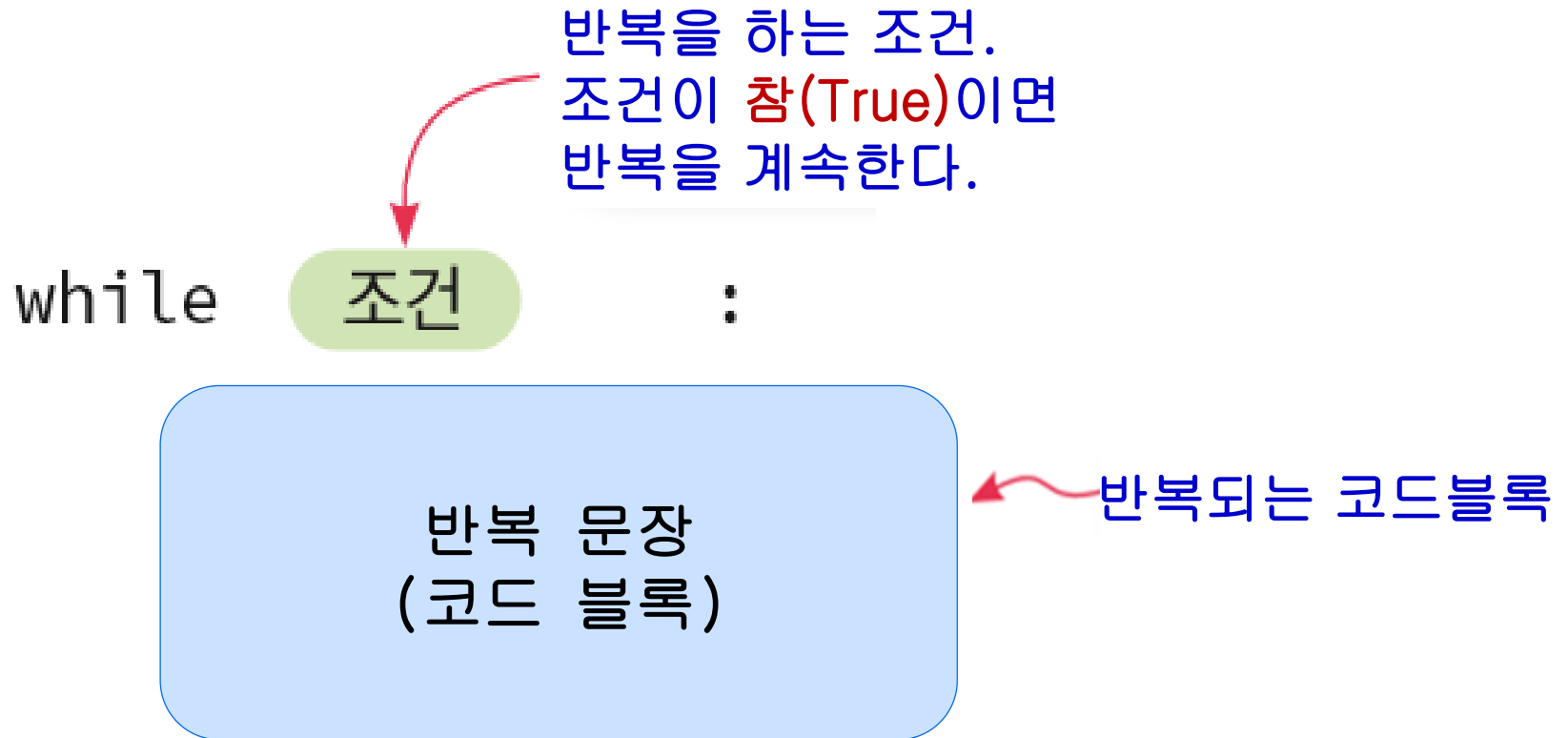


(출처: 어서와 파이썬은 처음이지)

- (비교) for Statement
 - for loop는 일정 횟수 만큼 반복 작업에 적당하다.

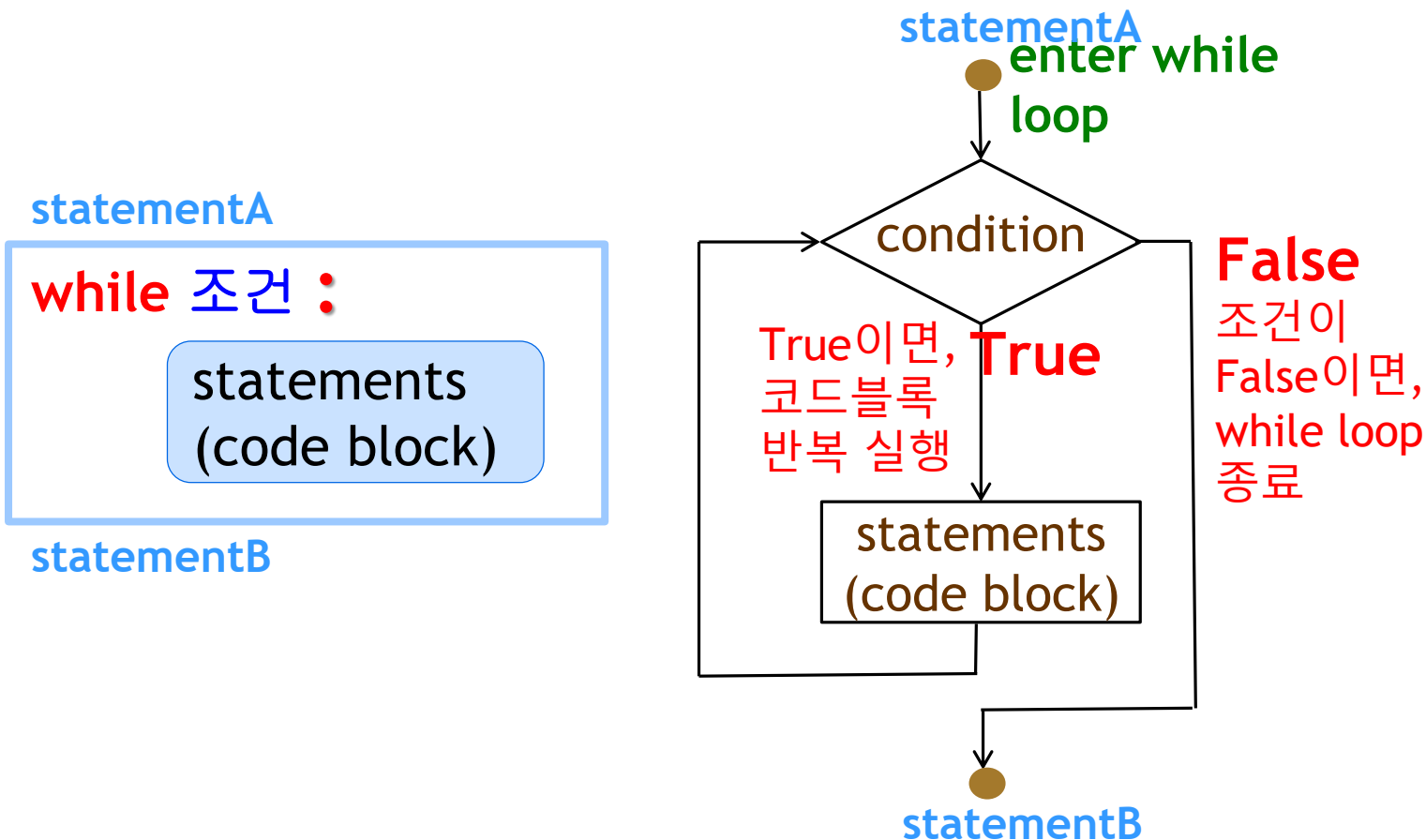
while 문

- while 문의 구조
 - 조건이 참(True)이라면 반복하는 구조.
 - 조건이 False가 되면 반복을 종료



while 문

- while 문은 주어진 조건이 True인 동안 loop안의 statement들(코드 블록)을 반복 수행하도록 해준다.



- while문은 조건에 따른 반복. 나머지 사항은 for문과 같다.

while 문

- 예) 1부터 10까지의 합계 구하기. $a = 1+2+3+4+5+....+9+10$
 - 변수 i 를 반복문안에서 1씩 증가 시킴.

```
a = 0 # 초기화. 합계 변수
i = 1 # 초기화 1부터 10까지 차례로 증가

while i <= 10 :
    a = a + i
    i = i + 1 # 다음반복을 위해 +1
print(a) # 55
```

	i의 값	a의 값
1번째 반복	1	0+1
2번째 반복	2	0+1+2
3번째 반복	3	0+1+2+3
...	...	...
10번째 반복	10	0+1+2+3+ +9+10

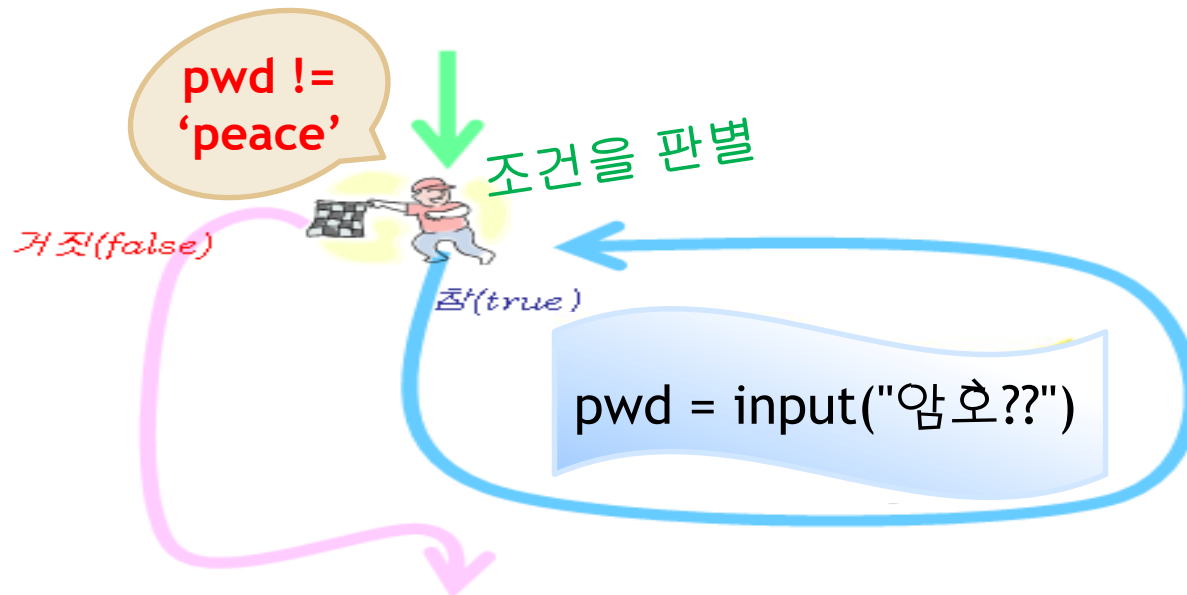
```
a=0
a=a+1
a=a+2
a=a+3
a=a+4
a=a+5
a=a+6
a=a+7
a=a+8
a=a+9
a=a+10
```

a = a + i

while 문

- 예) 암호가 맞지 않으면, 암호 입력을 다시 반복한다.
(패스워드는 'peace'라고 가정)

```
pwd = input("암호를 입력하시오: ")  
  
while pwd != "peace": #암호가 맞지 않으면 반복.  
    pwd = input("암호??")  
  
print("로그인 성공")
```



while 문

- 예) 사용자가 암호를 입력하기. 맞는지 확인하고, 맞을 때까지 암호를 다시 입력하도록 한다. (암호는 'peace'라고 가정)

실행 화면

암호를 입력하시오: idontknow 
암호를 입력하시오: 12345678 
암호를 입력하시오: peace 
로그인 성공

#비교

```
password = input("암호를 입력하시오: ")  
  
if password != "peace":  
    password = input("암호를 입력하시오: ")  
if password != "peace":  
    password = input("암호를 입력하시오: ")  
:  
:
```

if문을
몇 번 사용해야
할까요?

while 문

출력 화면

```
암호를 입력하시오: idontknow Enter
암호를 입력하시오: 12345678 Enter
암호를 입력하시오: peace Enter
로그인 성공
```

```
password = input("암호를 입력하시오: ")
```

```
while password != "peace": #암호가 맞지 않으면 반복
    password = input("암호를 입력하시오: ")
```

```
print("로그인 성공")
```

#또는

```
password = "?" #반복문의 조건이 True가 되도록 할당.
               # 암호가 아닌 값으로 할당하면 됨
```

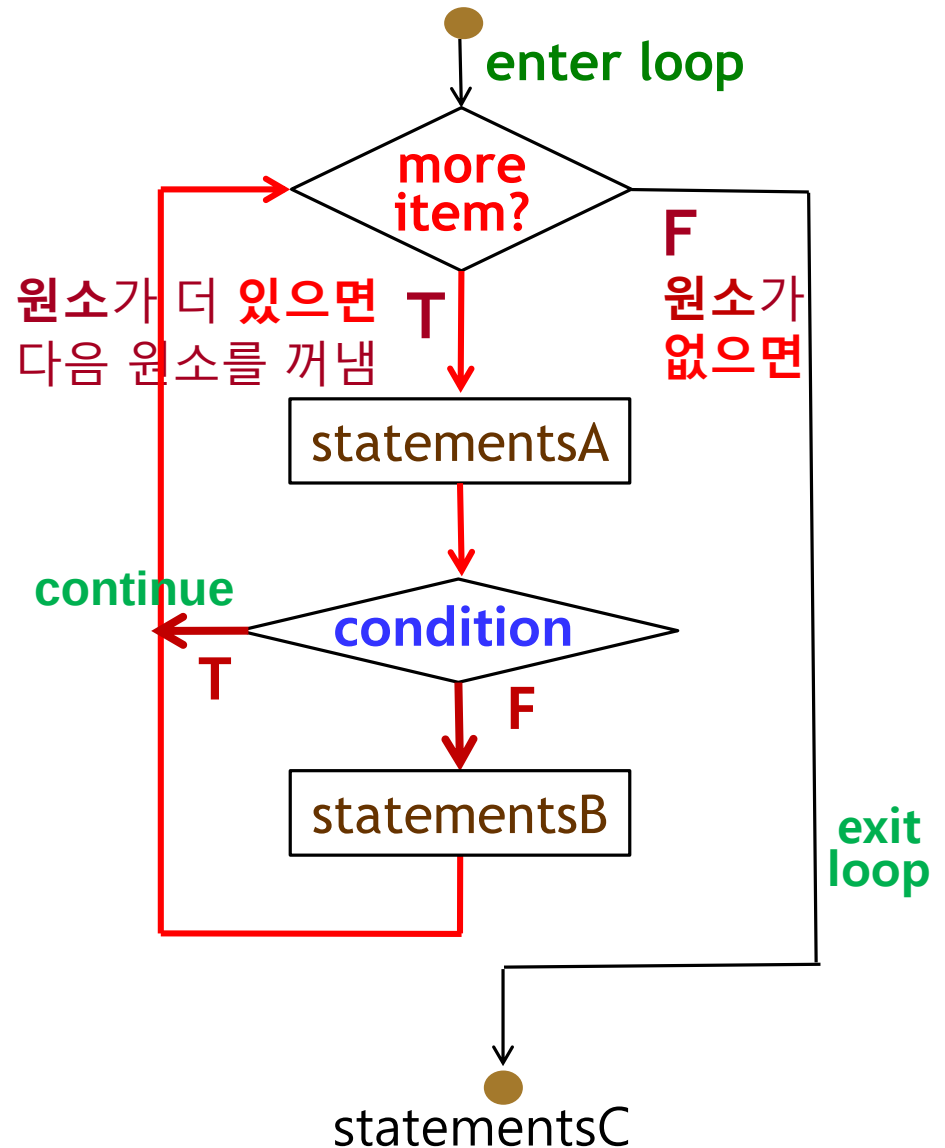
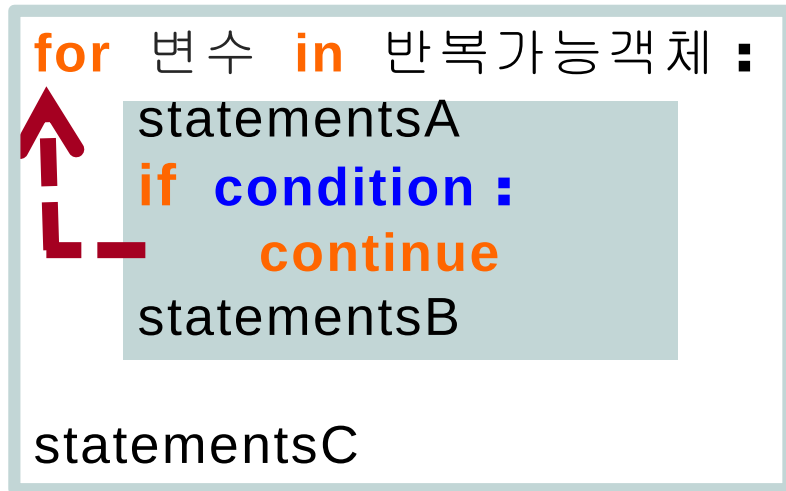
```
while password != "peace": #암호가 틀리면 반복. 다시 암호를
    password = input("암호를 입력하시오: ")    입력하도록 함
```

```
print("로그인 성공")
```

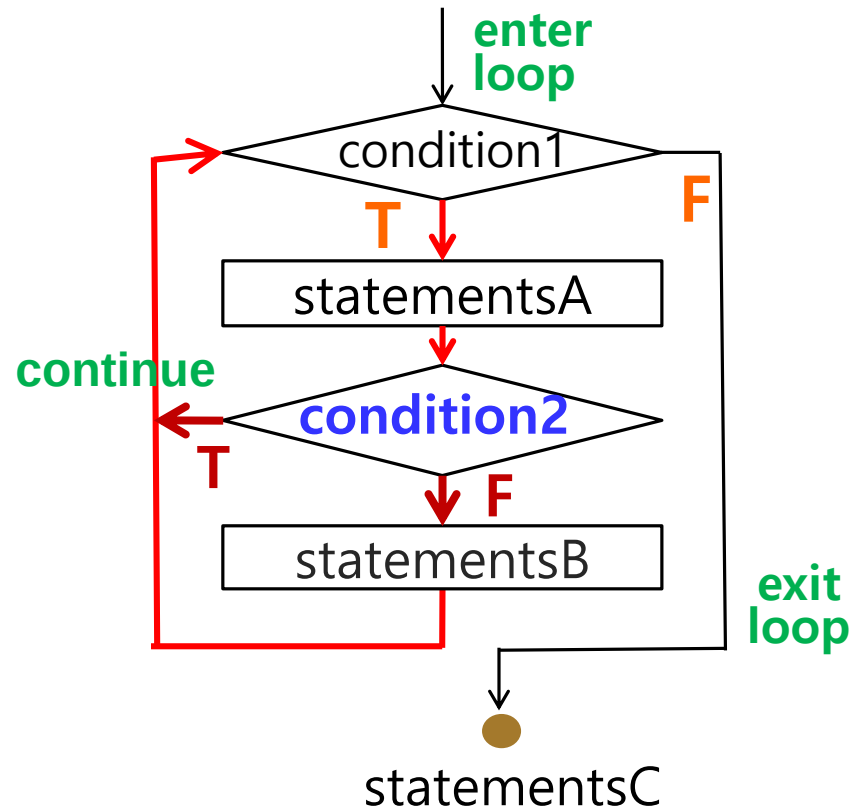
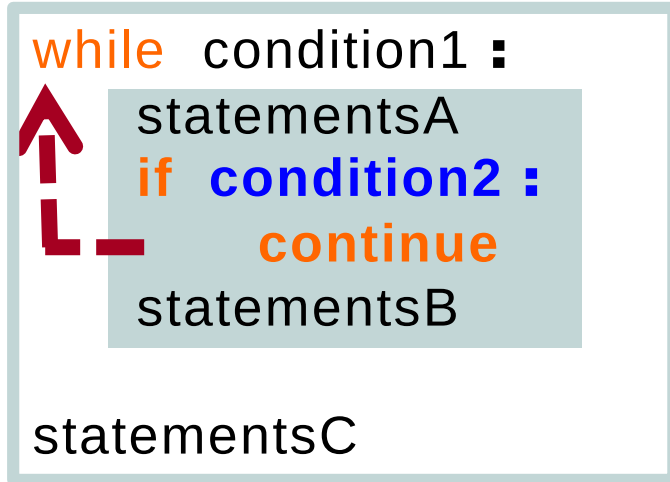
continue Statement

- 반복문의 코드블록안에서,
continue문 뒤에 문장들을 실행하지 않고 **skip**,
다음 반복을 위해 **loop**의 처음으로 돌아간다.
- 조건문과 함께 사용한다.
- 반복문의 코드블록에서만 사용 가능.
 - 반복문 안에서 **continue**를 실행할 조건을 **if**문으로 만든다.
 - **if**문의 **condition**이 **True**이면,
continue를 실행하여 **loop**의 처음으로 돌아간다.
 - **continue**전에 뭔가 할 일이 있으면, 그 앞에 **statement**를 추가한다.

continue Statement



continue Statement



continue Statement

- 예) 문자열을 받아서 모음을 전부 없애는 코드 작성하기
 - 모음이면, 새문자열에 연결하기를 skip하도록 한다.

```
s = input('문자열을 입력하시오: \n ')
vowels = "aeiouAEIOU"
result = "" # 빈 문자열로 초기화
for letter in s:
    #반복코드
    if letter in vowels: continue
    result = result+letter #연결하기
print(result)
```

조건이 참이면(모음이면)
연결하기를 skip하고,
for 문장으로 가서
다음 반복을 한다.

출력 화면

문자열을 입력하시오:
Obey To The Truth.
by T Th Trth.



while 문

- 예) 1부터 100에서 3의 배수를 제외한 수들의 합을 계산하기
 - 3의 배수이면, 누적 계산하기를 skip

```
hap = 0    # 합계를 담을 변수를 0으로 초기화
i = 1      # 카운터용 변수 초기화.
           # 반복문 안에서 +1시키며, 1부터 100까지 정수를 누적함.
```

```
while i <= 100 :
    if i % 3 == 0 :
        i = i+1 #다음 반복을 위해 +1
        continue
    hap = hap+i #누적 계산하기
    i = i+1    #다음 반복을 위해 +1
```

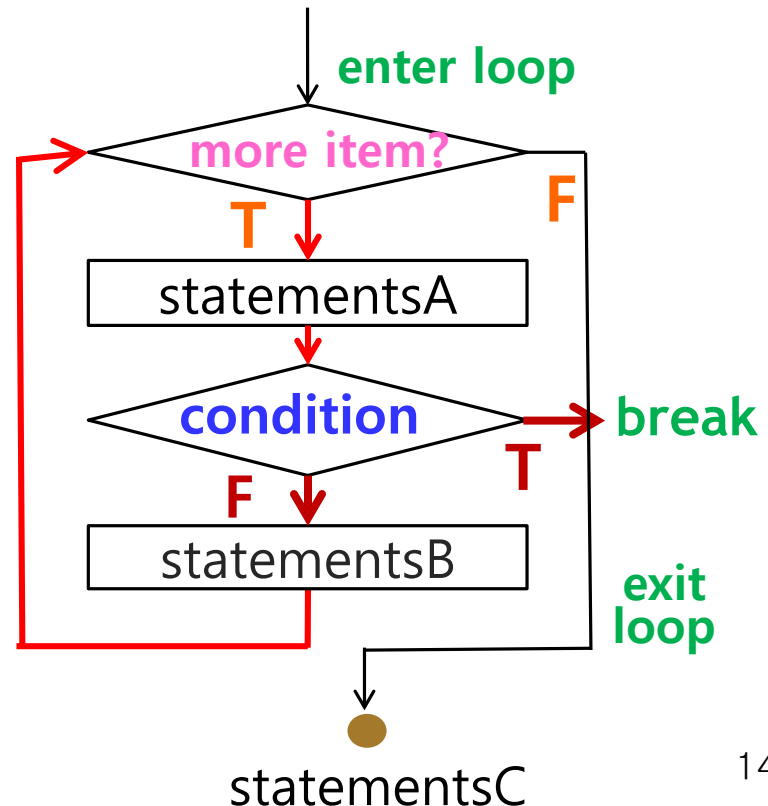
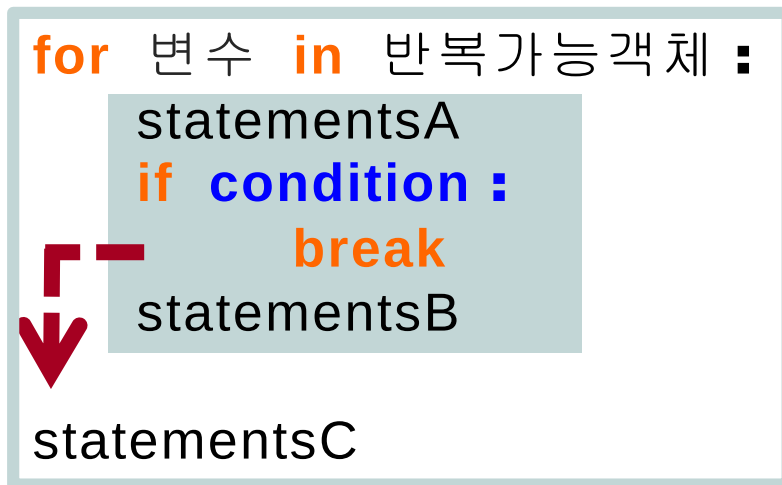
```
print("합계는 {}".format(hap) )
```

😊 출력 화면

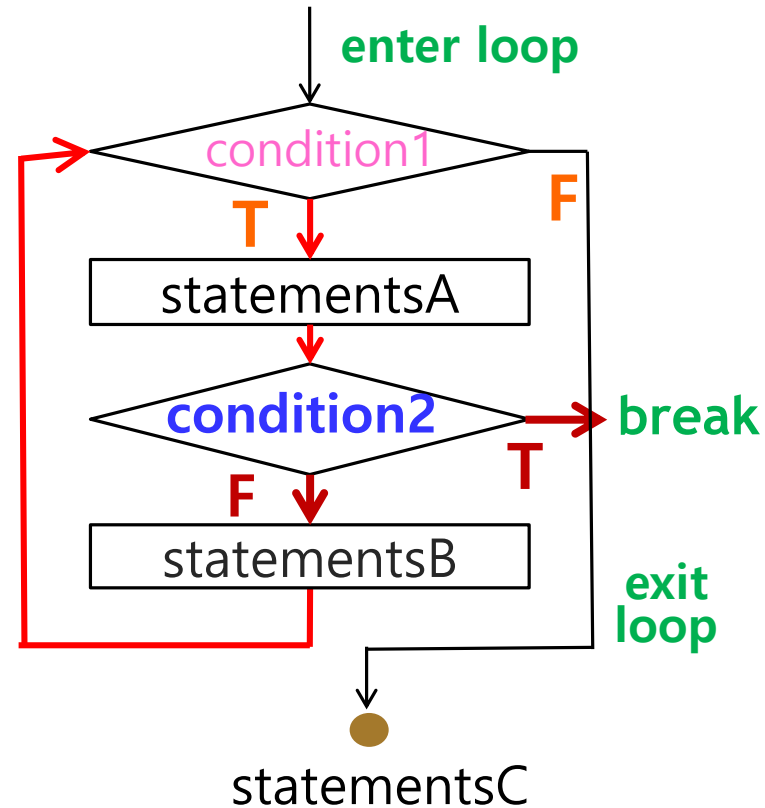
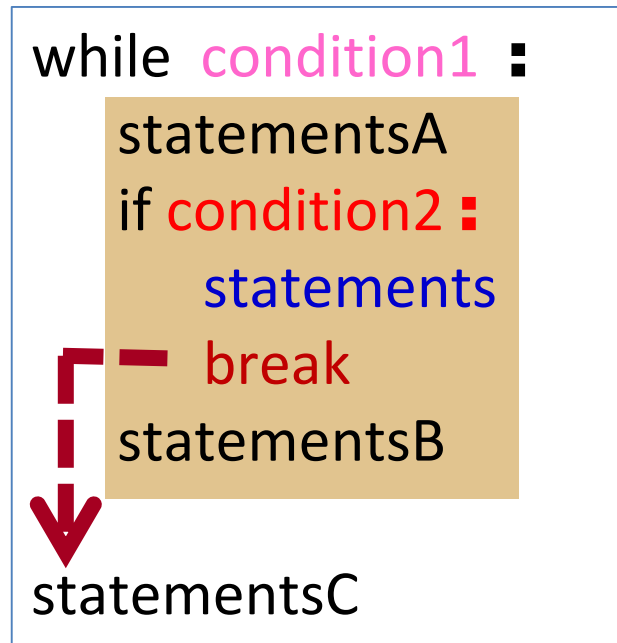
합계는 3367입니다.

break Statement

- 반복문(loop)을 강제로 종료한다.
보통 조건문과 함께 사용되며, 조건이 **True**이면 반복문을 종료.
- 프로그램 흐름 (반복문의 코드블록에서만 사용 가능)
 - if문장의 **condition**이 **True**이면 **loop** 밖으로 빠져 나간다.
 - break** 전에 뭔가 할 일이 있다면, 그 앞에 **statement**를 추가한다.



break Statement



break Statement

- break

- 반복문(loop)을 종료시키는 기능을 한다.
주로 조건문과 함께 사용되며, 조건이 맞으면 반복문을 중단시킴.
- 예) 리스트에 있는 원소들을 차례로 더하다가
합계가 100을 넘으면 멈춘다. (i가 14일때 반복을 종료함.)

```
hap = 0
num= list( range(1,100) ) # num=[1,2,3,...,99]

for i in num :
    hap = hap+i
    if hap > 100:
        print("반복을 종료해요")
        break # for문 종료

print(hap) # 105. hap= 1+2+3+4+ ...+14 = 105
```

Diagram illustrating the execution of the code block. A red arrow points from the variable `i` in the `for` loop to a sequence of numbers 1, 2, 3, ..., 99. A blue bracket groups the code block (the `for` loop and the `break` statement) and is labeled "for문의 코드블록". A red arrow points from the `break` statement to the `print(hap)` statement.

#비교

```
hap = 0

for i in range(1,100):
    hap = hap+i
    print(hap)
```

출력 화면

```
반복을 종료해요
105
```

break Statement

- 무한루프와 break

while(True) :

- while 루프의 조건(condition)을 True로 한다.
- 조건이 True이므로 무한히 반복된다.
- 따라서, 어떤 조건이 성립하면 무한 루프를 종료하도록 구성한다.
 - if 문장을 사용하여 특정조건에서 종료하도록 구성한다.
 - break문을 사용해서 루프를 빠져 나오도록 한다.

while 문

- 예) 사용자가 암호를 입력하기. 맞는지 확인하고, 맞을 때까지 암호를 다시 입력하도록 한다. (암호는 'peace'라고 가정)

출력 화면

```
암호를 입력하시오: idontknow Enter  
암호를 입력하시오: 12345678 Enter  
암호를 입력하시오: peace Enter  
로그인 성공
```

```
while True :  
    { password = input("암호를 입력하시오: ")  
      if password == "peace": break #암호가 맞으면 반복 종료  
    print("로그인 성공")
```

break Statement

- 예) 입력 값을 받고 출력하기.
 - 입력 값을 받고, 그 값을 출력한다.
 - 만약 입력 값이 0이면 반복을 종료.
 - 출력하기를 하지 않고 종료. break문을 사용

```
while (True) :  
    N = int( input('숫자 입력? ') )  
    { if N==0 : break  
      print(N) #출력하기  
  
print("bye")
```

비교

```
while (True) :  
    N = int( input('숫자 입력? ') )  
    if N==0 : break  
    else: print(N)  
  
print("bye")
```

출력 화면

```
숫자 입력? 10 Enter  
10      # 입력 값 출력  
숫자 입력? 20 Enter  
20  
.....  
.....  
숫자 입력? 0 Enter 출력 X  
bye
```

while 문

- 예) 사용자가 입력한 숫자를 더하는 프로그램
 - 입력 값(정수)을 받아들이고 합계를 구함.
 - 음수를 입력할 때 까지 반복을 계속한다. **합계에 포함X.**
break문을 사용

😊 출력 화면

종료하려면 음수를 입력하십시오

숫자를 입력하십시오: 10 

숫자를 입력하십시오: 20 

숫자를 입력하십시오: 30 

숫자를 입력하십시오: -1  **합계에 포함 X**

합계는 60입니다.

while 문

```
# 필요한 변수를 초기화
total = 0 # 누적 합계를 위한 변수

print("종료하려면 음수를 입력하십시오")

while True: # 숫자를 입력받아 누적하기
    score = input("숫자를 입력하십시오: ")
    score = int(score)
    if score < 0: break # 음수가 입력되면, 누적 X. 반복을 종료
    total = total + score # 누적 합계 구하기

# 출력한다.
print( "합계는 {}입니다.".format(total))
```